

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

Snowflake Data Cloud Features and Architecture

What is Snowflake?

- Cloud based data platform (AWS, GCP, Azure Supported)
- Not built from old db tech
- Cloud native
- Separates Compute from Storage
- It is SaaS (Min upfront cost, Pay-as-you-go, fully managed Infra)

Traditional Architecture:

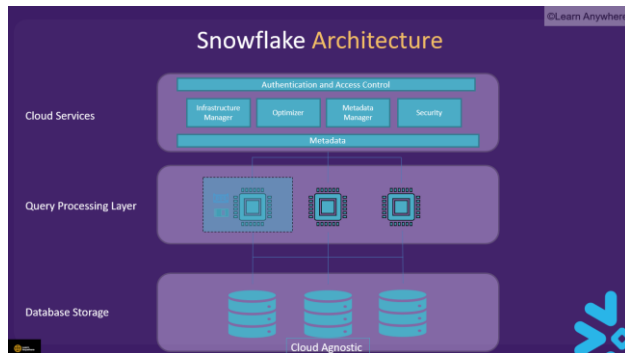
Shared Disk Architecture (ex: Oracle Real Application Clusters)

- High availability/reliability, easy data sharing b/w nodes, dynamic load balance
- **Single point of failure, Network bottleneck**

Shared Nothing Architecture (ex: Teradata)

- High scalable, better performance, flexible conf
- **Fixed load balancing, Data Skew, Time consuming data partitioning**

Snowflake (SF) Architecture: SF has 3 distinct layer, Cloud Services, Query Processing Layer, Database Storage.



Database Storage:

- Data into optimized, Compressed and columnar format (Proprietary)
- Data is stored in cloud storage, accessible to all nodes
- Data not accessible directly, need to read via SF
- Cloud agnostic (AWS, GCP, Azure Supported)

Query Processing Layer:

- Data into optimized, Compressed and columnar format
- Data is stored in cloud storage, accessible to all nodes
- Data not accessible directly, need to read via SF
- Cloud agnostic (AWS, GCP, Azure Supported)

Cloud Services:

- Snowflake cloud services coordinate platform activities
- Manage user requests from login to query dispatch
- Responsible for security, access management, monitoring and other operational tasks

- **Authentication and access control** verifies user identity and manages access to data.
- **Infrastructure manager** provisions and manages compute resources for Snowflake.
- **Snowflake optimizer** determines the most efficient way to execute a query.
- **Metadata Manager** manages the metadata associated with data stored in Snowflake.

- **Snowflake Security** includes encryption, network isolation, and other security features to protect data.

Snowflake has multiple editions to choose from:

Standard	Enterprise	Business Critical	CLearn Anywhere
All Standard Features	All Standard Features	All features of Enterprise	Most features of Business Critical
	Multi Clustered VW	Fallover and DR	Highest level of Security
	Materialized View, Search Optimization	Redirecting Client Connection	Separate Environment
	Extended Time Travel	HIPAA, HITRUST CSF Environment	

For each Snowflake account, you choose **Edition, cloud provider and location/Region**. Snowflake UI is called Snowsight.

Snowflake Partners: Huge partner network for data integration, ML/Data Science, Security, Governance & Observability, BI, SQL Dev & Mgmt and Native programming Interface.

Snowflake Drivers and Connectors: Supports Go, JDBC, ODBC, .Net, Node.js, PHP PDOst, Python, Kafka Connector, Apache Spark.

SQL Scripting: Snowflake Scripting is an extension to Snowflake SQL that adds support for procedural logic.

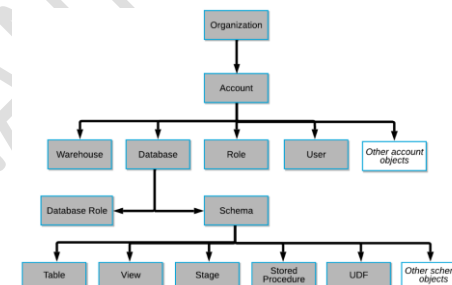
You can Declare Variables, return a value, branching construct, IF/CASE, Loops, FOR, WHILE, REPEAT, LOOP, CURSOR, RESULTSET, determine number of rows impacted by DML (SQLROWCOUNT, SQLFOUND, SQLNOTFOUND), Handles Exception.

Snowpark: Snowpark is a developer experience for Snowflake that allows developers to write code in their preferred language (Python, Scala, or Java) and run that code directly on Snowflake.

SnowCD (Snowflake Connectivity Diagnostic Tool) helps users to diagnose and troubleshoot their network connection to Snowflake.

SnowCD leverages IP Listed by

SYSTEM\$ALLOWLIST()
SYSTEM\$ALLOWLIST_PRIVATELINK()



Database: A database is a logical grouping of schemas. Each database belongs to a single Snowflake account.

Schema: A schema is a logical grouping of database objects (tables, views, etc.). Each schema belongs to a single database.

Transient Schema doesn't have fail-safe option, tables under this are transient as well. **Managed Access Schema** schema owner manages all privilege grants.

Table Types :

Features	Permanent	Transient	CLearn Anywhere
Persistence	Until Dropped	Until Dropped	Session
Cloning	To Any type	To Transient or Temporary	To Transient or Temporary
Time Travel (Days, Default is 1)	0-1 (0-90)	0-1	0-1
Fail Safe (Days, Fixed)	7	0	0

* External table, none of the above criteria applies.

*External tables are read only and points to external stage, There are more tables like Dynamic, Event, Iceberg and Hybrid

External Table : Read only table, outside of SF, Slower than native tables, partition can enhance performance. Includes columns: **VALUE, METADATA\$FILENAME, METADATA\$FILE_ROW_NUMBER.**

Code for table creation:

```
CREATE TEMPORARY (TEMP)| TRANSIENT| PERMANENT (Optional) |EXTRNAL TABLE  
<Table_name> (  
  ID NUMBER,  
);  
Added for External>>, With LOCATION = .., PATTERN = ...,FILE_FORMAT = ..
```

Data Types:

Data Types					
Numeric	String/Binary	Logical	Date & Time	Geospatial	Semi-Structured
- Number - Decimal - Numeric - INT, FLOAT, - DOUBLE	- VARCHAR - CHAR - STRING,TEXT - BINARY	- BOOLEAN	- DATE - DATETIME - TIMESTAMP	- GEOGRAPHY - GEOMETRY	- VARIANT - OBJECT - ARRAY

View Type:

	Non-Materialized View	Materialized View	Secure View
Definition	A virtual table based on the result of a SELECT statement.	A pre-computed and stored table based on the result of a SELECT statement.	Non-Materialized and Materialized view can be set as Secure View
Storage	Data is not stored, it is computed on the fly each time the view is queried.	Data is stored in the cache, allowing for faster query execution.	Same as underlying View.
Use Case	Data transformation, providing a simplified view of the underlying data.	Reporting and analytics, frequently queried data.	When you want to limit access to the data

Code for View Creation :

```
CREATE OR REPLACE SECURE MATERIALIZED VIEW <View_name> AS SELECT....
```

Advantage of Materialized View: **Enterprise + feature**

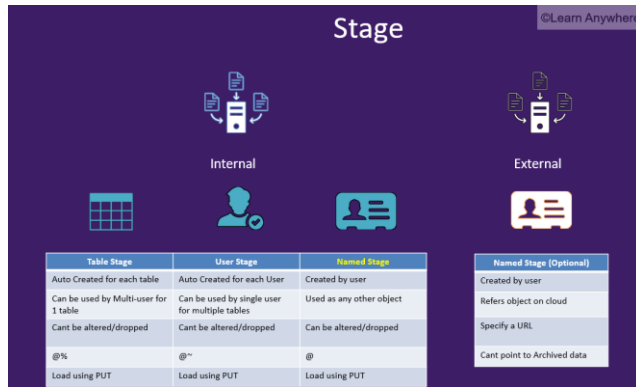
- Materialized views can improve the performance of repeat queries.

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

- Materialized views are automatically by Snowflake.
- Data accessed through materialized views is always Up-to-date.

Extra cost of Storage and compute (Managed by SF)

Stage : “Snowflake refers to the location of data files in cloud storage as a stage”



UDF and UDTF:

“With user-defined functions (UDFs), you can extend the system to perform operations that are not available through the built-in, system-defined functions provided by Snowflake.”

UDF with returning Table is called UDTF.

Languages Support: SQL, JS, Python, Java

Example: `SELECT demodb.demoschema.udf_sample()`

Stored Procedure:

A stored procedure is a named, precompiled collection of SQL statements and control-of-flow language constructs that are stored in a database and can be called from SQL CALL command, including other stored procedures.

Languages Support: SQL, JS, Python (These 3 via Snowpark), Java, Scala

UDF Vs SP:

Feature	User-Defined Function (UDF)	Stored Procedure
Return Value	Must return a value	Not Mandatory to return a value
Use case	When part of SQL stmt	For DML, DDL Operations
Languages	Java, JS, Python, Scala, SQL	Java, JS, Python, Scala, Snowflake Scripting
Granularity	Can be called for every row of a query result set	Can only be executed as a standalone command
Inside SELECT	Multiple UDFs can be called	Single SP with a statement, Can have nested SP
DB Access	May not have access	Has access

Sequences:

Sequences are used to generate unique numbers across sessions and statements; however, Snowflake doesn't guarantee generating sequence numbers without gaps.

Syntax : `CREATE OR REPLACE SEQUENCE SEQUEN_EX START = 1 INCREMENT = 1;`

`SELECT SEQUENCE_SAMPLE2.nextval a, SEQUENCE_SAMPLE2.nextval b, SEQUENCE_SAMPLE2.nextval c, SEQUENCE_SAMPLE2.nextval d;`

- Default value for the INCREMENT and START is 1.

Micro-Partitioning:

- Snowflake automatically divides table in micro-partitions
- Each micro-partition contains 50-500 MB of uncompressed data (SF compress it further and stores)
- Organized in columnar format
- Very large tables can have multi-million micro-partitions
- Micro-partitions are immutable (Time-Travel, Cloning)

Snowflake stores MP metadata:

- Range of values for each of the column in MP
- Number of distinct values
- Additional info which can be used for optimized and efficient query processing

Clustering

Clustering is a technique by which Snowflake co-locate same value data in same micro-partitions, to improve the performance and optimize the cost.

How to choose candidate for Clustering:

- Table with large number of Micro-partitions
- Queries need to read small amount of data (WHERE clause)
- Queries Sort the data (Order by)
- Many/most queries do select on, or sort on, the same few column(s)

Snowflake maintains clustering metadata

- Total Number of Micro-Partitions
- Number of micro-partitions containing values that overlap with each other
- Depth of the overlapping partitions

Overlapping MP and Overlap Depth



SYSTEM\$CLUSTERING_DEPTH - Computes the average depth of the table according to the specified columns.

SYSTEM\$CLUSTERING_INFORMATION (including clustering depth) - Returns clustering information, including average clustering depth, for a table based on one or more columns in the table

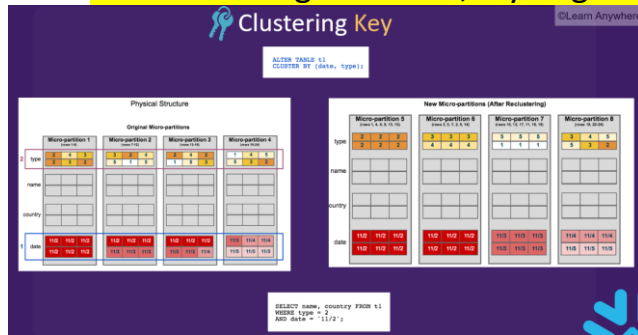
Code for Clustering :

`ALTER TABLE <table_name> CLUSTER BY (<column>);`

How to choose Clustering Keys :

- Use column most used in where clause
- Choose Join predicates
- Balance out the cardinality (Lower >> minimal pruning, Higher >> Inefficient grouping)
- For Multi Clustering Key move from low to high cardinality

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>



Data storage monitoring

Select Admin » Cost Management » Consumption

Snowflake provides various methods to view and control data storage for tables, stages, and clones. Users can use Snowsight, the Classic Console, the SHOW TABLES command, or the TABLE_STORAGE_METRICS view to monitor storage usage and costs. Users can also remove staged files or drop cloned objects to reduce storage.

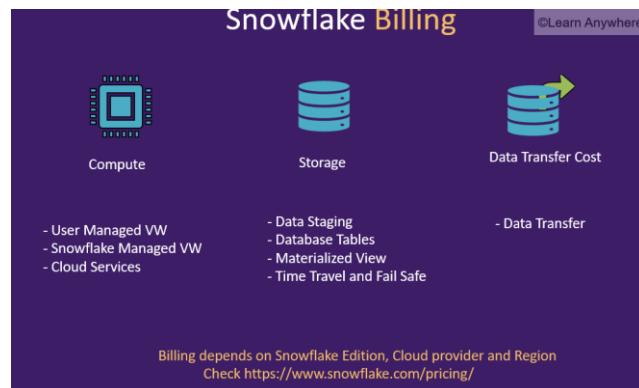
--Local Schema

```
SELECT * FROM LA_DB.INFORMATION_SCHEMA.TABLE_STORAGE_METRICS;
```

-- Across Account

```
SELECT * FROM  
SNOWFLAKE_SAMPLE_DATA.INFORMATION_SCHEMA.TABLE_STORAGE_METRICS;
```

Snowflake Billing :



Compute Cost :

- A virtual warehouse is one or more clusters of compute resources that enable executing queries, loading data, and performing other DML operations.
- Snowflake credits are used to pay for the processing time used by each virtual warehouse.

Type of Virtual Warehouse	X-Small	Small	Medium	Large	X-Large	2X-Large	3X-Large	4X-Large
Standard	1	2	4	8	16	32	64	128
Snowpark-optimized	n/a	n/a	6	12	24	48	96	192

Cost Based on Size of VW and How long they Run, not on complexity of the Query.

- Billed By Seconds
- 60 Sec minimum
- When Scale Up, Charged only for extra compute
- When Scale Down, Charged for both, for brief period

Compute Cost for Cloud services :

- Authentication, enforces security, performs query compilation and optimization, handles request query caching etc.
- Snowflake credits are used to pay for the processing time used by each virtual warehouse.
- If usage is less than 10% of daily credit consumed, it is free.

Snowflake Storage Cost:

- Staged File Costs
- Database Costs
- Time Travel and Fail-safe Costs
- Temporary and Transient Tables Costs
- Cloning Tables, Schemas, and Databases Costs

Snowflake Data Transfer Cost : Data transfer is the process of moving data into (ingress) and out of (egress) Snowflake.

- **Unloading** - No charges for data ingress/loading but cost if you unload
- **Replication** - Data transfer in same region and same cloud provider is free Cost when region or cloud provider is different
- **Writing External Function** - When data gets transferred out for function
- Per-Byte fee for Egress

**Snowflake does not apply data egress charges when a Snowflake client or driver retrieves query results across regions within the same cloud platform or across different cloud platforms*

Features	On Demand	Capacity
Billing	Pay-as-you-go	Reserve in advance
Rates	Prevailing prices	Discounted prices

Account Access and Security

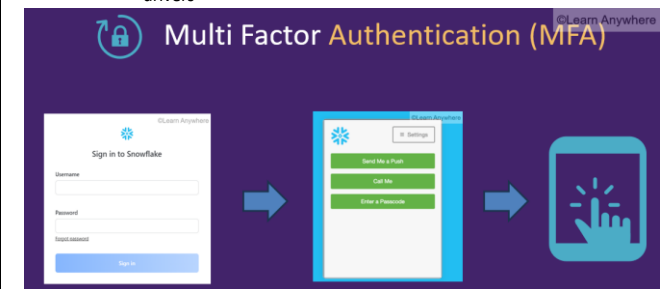
Network security and policies (Admin » Security » Network Policies)

- Allows to restrict access to SF based on IP address
- SECURITYADMIN or Higher (eg ACCOUNTADMIN) can create network policies
- Can be set at ACCOUNT or USER level
- Only one Policy per ACCOUNT, or per USER
- Supports CIDR Notation
- Blocked IP gets preference
- Support only Internet Protocol version 4 (i.e. IPv4) addresses, not IPv6

```
CREATE NETWORK POLICY mypolicy1  
ALLOWED_IP_LIST=('192.168.1.0/24')  
BLOCKED_IP_LIST=('192.168.1.99');
```

Multi-Factor Authentication (MFA) : Powered By Duo Security Service

- Managed by Snowflake, User can self enroll
- Not enabled by default
- Recommended to have MFA for all ACCOUNTADMIN role
- Ideally for Snowsight but fully supported for Snowsql, JDBC/ODBC drivers



Federated Authentication (SSO) : In a federated environment, user authentication is separated from user access through the use of one or more external entities that provide independent authentication of user credentials.

Service Provider:

In a Snowflake federated environment, Snowflake serves as the SP.

Identity provider (IdP):

Creating and maintaining user credentials and other profile information.

Authenticating users for SSO access to the SP.

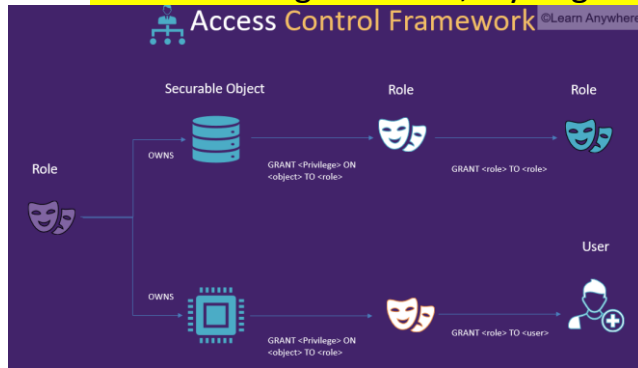
Snowflake Supports SCIM 2.0, Native app support : Microsoft and Okta

Access Control Framework (DAC + RBAC)

Discretionary Access Control (DAC) : Each object has an owner, who can in turn grant access to that object.

Role-based Access Control (RBAC) : Access privileges are assigned to roles, which are in turn assigned to users.

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>



Securable object: An entity to which access can be granted. Unless allowed by a grant, access is denied.

Role: An entity to which privileges can be granted. Roles are in turn assigned to users. Note that roles can also be assigned to other roles, creating a role hierarchy.

Privilege: A defined level of access to an object. Multiple distinct privileges may be used to control the granularity of access granted.

User: A user identity recognized by Snowflake, whether associated with a person or program.

Account roles

To permit SQL actions on any object in your account, grant privileges on the object to an account role.

Database Roles (Preview)

To limit SQL actions to a single database, as well as any object in the database, grant privileges on the object to a database role in the same database.

Primary Role

The default assumed Role `USE ROLE <ROLE NAME>`

Secondary Role

Can assign any number of secondary role

Privileges are granted to roles, and roles are granted to users, to specify the operations that the users can perform on objects in the system.

-- Grant ROLE To new user

```
GRANT ROLE DUMMY_ROLE TO USER USER1;
```

-- Grant SELECT on TABLE

```
GRANT SELECT ON TABLE DUMMY.DUMMY_SC.cust_det TO ROLE DUMMY_ROLE;
```

-- USAGE on DB and SCHEMA Needed in order to access TABLE

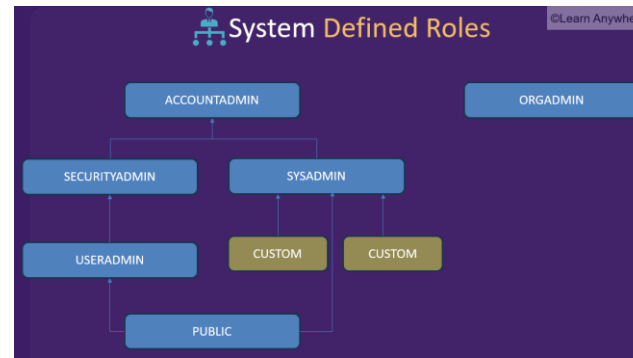
```
GRANT USAGE ON DATABASE DUMMY TO ROLE DUMMY_ROLE;
```

```
GRANT USAGE ON SCHEMA DUMMY.DUMMY_SC TO ROLE DUMMY_ROLE;
```

Type of table privilege

`SELECT`
`TRUNCATE`
`DELETE`
`OWNERSHIP`
Etc...

System Defined and Custom Roles



ORGADMIN : At Org level, Can create Account, can run `SHOW ORGANIZATION ACCOUNTS`, `SHOW REGIONS`

ACCOUNTADMIN : Parent to SYSADMIN and SECURITYADMIN

SECURITYADMIN : Manages GRANTS, Manage USERS/ROLES, PARENT OF USERADMIN

SYSADMIN : Create DB, WH, Assign all custom role to SYSADMIN

USERADMIN : CREATE USER/ROLE

PUBLIC : Pseudo Role, Auto granted to all

Role Hierarchy and Privilege Inheritance:



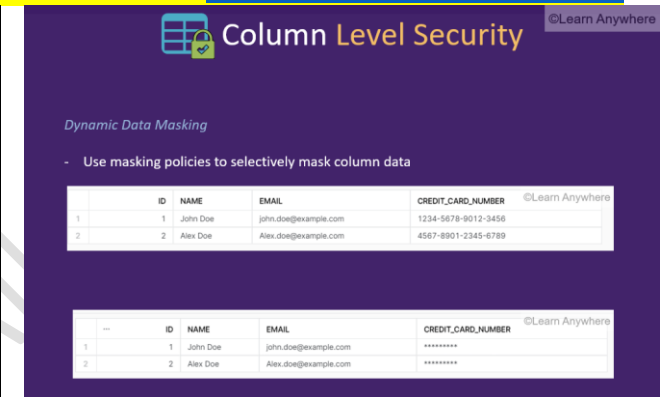
Column Level Security : Enterprise + feature

Dynamic Data Masking (Needs Enterprise Edition or Higher)

- Use masking policies to selectively mask column data

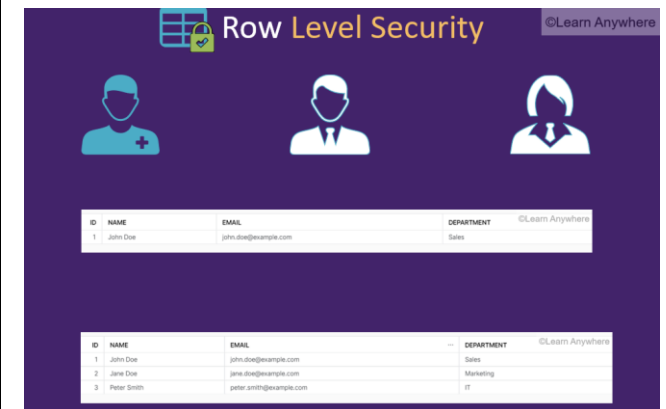
External Tokenization

- Tokenization is the process of removing sensitive data by replacing it with an undecipherable token



Row Level Security : Enterprise + feature

Only relevant rows are shown to authorized person



An **organization** is First-class Snowflake object that links the accounts owned by the org. Organizations simplify Account management and billing, Replication & Failover/Failback, Snowflake Secure Data Sharing and administration tasks.

- Use ORGADMIN Role

Account identifier uniquely identify the SF account:

Format: `<account_identifier>.snowflakecomputing.com`

`<account_identifier> = <orgname>-<account_name>`

Information Schema:

The Snowflake Information Schema (aka "Data Dictionary") consists of a set of system-defined views and table functions that provide extensive metadata information about the objects created in your account. The Snowflake Information Schema is based on the

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

SQL-92 ANSI Information Schema, but with the addition of views and functions that are specific to Snowflake.

- Read Only Schema
- Available for every DB
- Maintains data about all DB object and some account level information, In form of Views
- Table function for historical and usage data across your account
- Need Privilege

-- Find the login history with information schema

```
SELECT * FROM TABLE(information_schema.login_history());
```

Account Usage:

Views that display object metadata and usage metrics for your account.

- Type: Account_Usage and Readers_Account_Usage

Parameters	Account Usage	Information Schema
Includes dropped object	Yes	No
Latency of data	45 Mins – 3 Hours	Instant
Retention Period	1 Year	7 Days to 6 Months

-- Find the login history at account level

```
SELECT * FROM account_usage.login_history;
```

Access History : **Enterprise + feature** Access History in Snowflake refers to when the user query reads data and when the SQL statement performs a data write operation, such as INSERT, UPDATE, and DELETE along with variations of the COPY command, from the source data object to the target data object.

- Needs Enterprise Edition or Higher
- Maintains history of read and write operations
- Used for compliance auditing
- Stores data on source column, object accessed directly and indirectly

Columns in Access_History

QUERY_ID | QUERY_START_TIME | USER_NAME | DIRECT_OBJECTS_ACCESSED |
BASE_OBJECTS_ACCESSED | OBJECTS_MODIFIED | OBJECT_MODIFIED_BY_DDL |
POLICIES_REFERENCED

Code example for access_history :

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY  
WHERE USER_NAME = 'TRAINING' ORDER BY QUERY_START_TIME;
```

Object Tags : Object tags in Snowflake are schema-level objects that allow you to attach metadata to other Snowflake objects, such as tables, views, and columns. Needs Enterprise Edition or Higher.

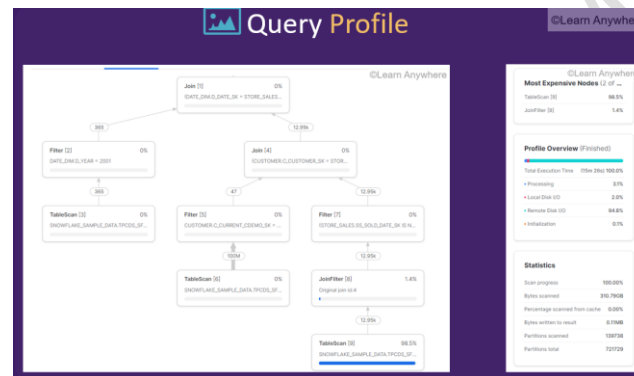
-- Create tag with allowed values

```
CREATE TAG IF NOT EXISTS COST ALLOWED_VALUES 'MKT', 'ADVT';  
CREATE TAG IF NOT EXISTS HR ALLOWED_VALUES 'ADMIN', 'TA';
```

Can also assign TAGS using Snowsight, Monitoring > Governance>Tagged Objects or under Database section.

Performance Concepts

Query Profile: It provides a graphical representation of the main components of the processing plan for the query, with statistics for each component, along with details and statistics for the overall query.



EXPLAIN PLANS

EXPLAIN [USING {TABULAR | JSON | TEXT}] <statement>

Query History (Option under Activity)

Provides 14 days history

QUERY_HISTORY (Under Information Schema)

Last 7 days activity

QUERY_HISTORY View (Under SF Account Usage)

Up to 365 days activity

Data Spilling: Data spilling occurs when an operation cannot fit in memory. The system begins by spilling data to the local disk of a warehouse node then to remote disks. These operations can significantly slow down query execution. To mitigate this, it's recommended to use a larger warehouse, process data in smaller batches, or optimize the query.

Types of Cache:

Query Result Cache (USE_CACHED_RESULT)

In Cloud services layer

Expires in 24 hours (Extendable up to 31 days)

Work for exact same query (not even case changes allowed)

Doesn't work with functions, which evaluates at runtime e.g. CURRENT_TIMESTAMP() and UUID_STRING(), CURRENT_DATE() is exception

Doesn't work with external functions

Doesn't work when, table change, config change, Micro-partitions change

Query result cache > 100KB uses security token

Security token expires in 6 hours (except for spark connector)

Doesn't work for Hybrid tables

Metadata Cache

In Cloud services layer

Stores Metadata Information

Warehouse not needed

Row Count, MIN/MAX values, DISTINCT count etc.

Warehouse Cache (Local Disk Cache)

At Query Processing Layer

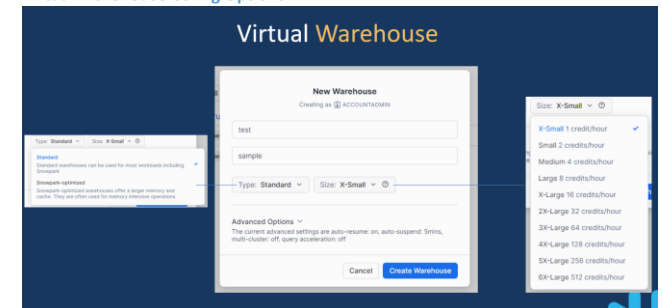
Local to the WH

Purged when WH is suspended or shut

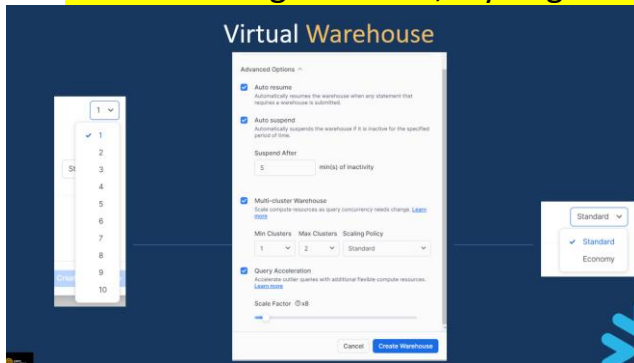
Need to find whether it is worth to auto-suspend a WH

Query Pruning: Skipping unnecessary scan of micro-partition is Query Pruning Improves the performance.

Virtual Warehouse Config Options :



This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>



Multi Cluster Warehouse:

Multi-cluster warehouses, Snowflake supports allocating, either statically or dynamically, additional clusters to make a larger pool of compute resources available.

Options available:

Maximum number of clusters, greater than 1 (up to 10).

Minimum number of clusters, equal to or less than the maximum (up to 10).

Additionally, multi-cluster warehouses support all the same properties and actions as single-cluster warehouses, including:

Specifying a **warehouse size**.

Resizing a warehouse at any time.

Auto-suspending a running warehouse due to inactivity; note that this does not apply to individual clusters, but rather the entire multi-cluster warehouse.

Auto-resuming a suspended warehouse when new queries are submitted.

Scale Out (Enterprise + feature) in Snowflake refers to the ability of Snowflake to automatically add or remove clusters (As per the demand) to a virtual data warehouse based on the query load or concurrent user.

Scale Up in Snowflake means increasing the resources for a single warehouse. It's like upgrading from a SMALL to a MEDIUM or from a MEDIUM to a LARGE warehouse.

Scale Down is exactly opposite of Scale up. Happens post completion of existing queries.

Scaling Policy:

Policy	Description	Warehouse Starts...	Warehouse Shuts Down...
Standard (default)	Prevents/minimizes queuing by favoring starting additional clusters over conserving credits.	The first cluster starts immediately when either a query is queued or the system detects that there's one more query than the currently-running clusters can execute.	After 2 to 3 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.
Economy	Conserves credits by favoring keeping running clusters fully-loaded rather than starting additional clusters, which may result in queries being queued and taking longer to complete.	Each successive cluster waits to start 20 seconds after the prior one has started. For example, if your warehouse is configured with 10 max clusters, it can take a full 200+ seconds to start all 10 clusters.	After 5 to 6 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.

You can monitor **Warehouse load** Admin>>Warehouse: You can check warehouse wise Query history.

Query Status	Description
Running	Queries that were actively running during the interval. Note that they may have started running before and continued running after the interval.
Queued	Queries that were waiting to run due to warehouse overload (i.e. waiting for other queries to finish running and free compute resources).
Queued (Provisioning)	Queries that were waiting while the warehouse provisioned compute resources. Typically only occurs in the first few minutes after a warehouse resumes.
Queued (Repairing)	Queries that were waiting while the warehouse repaired compute resources. Occurs only rarely and usually does not last for longer than a few minutes.

Resource Monitor:

- Monitoring credit usage by user managed WH and cloud services (Not Serverless)
- Can set at Account level or WH level
- Can set Credit Quota (Eg, 100 Credit)
- Monitor consumption by User managed WH and Cloud services
- You can define frequency at which monitoring resets
 - (Daily, Weekly, Monthly, Yearly, Never)
- Action on reaching threshold (Suspend Immediately, Suspend, Notify)

Query Acceleration Service (QAS) (Enterprise + feature)

- QAS can accelerate part of WH workload
- Reduces the impact of outlier query
- Works for (Ad hoc Analysis, Unpredictable data volume per query, Queries with large scan and selective filter)

SF Performance Optimization for below areas:

Storage: Clustering, QAS, SOS, Materialized View, Purge stage data, use of temp table, Save on time travel (DATA_RETENTION_TIME_IN_DAYS), Monitor memory consumption.

Warehouse : Reduce Queues, Reduce spilling, Right size WH, WH cache optimization, limit concurrent queries (MAX_CONCURRENCY_LEVEL), Separate WH , Monitor WH, Resource Monitor.

Query : Query Profile stat, Query History, Avoid SELECT *, Use filter.

Use of Specific SELECT Command

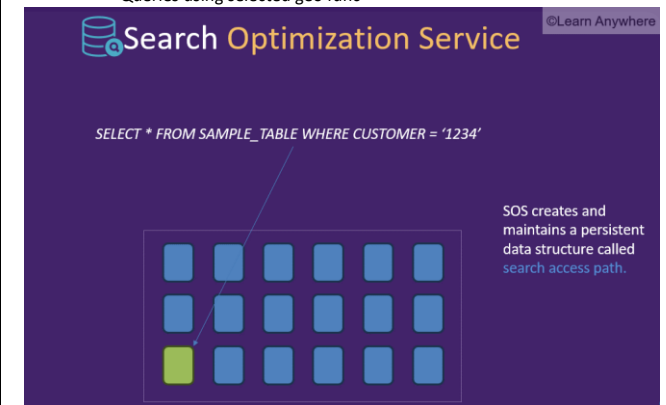
- Selecting only needed column in the query
- Use proper filtering
- Use windows function
- Use CTE (Common Table Expressions)
- Create Materialized View
- Use proper joining conditions
- Use clustering Key effectively
- Leverage Snowflake Cache

Search optimization service: (Enterprise + feature)

The search optimization service can significantly improve the performance of certain types of lookup and analytical queries that use an extensive set of predicates for filtering.

What Kind of Queries can be improved?

- Selective point look up queries, returns small distinct rows
- Substring or regular expression searches (LIKE, ILIKE etc.)
- Queries on fields in VARIANT, OBJECT, ARRAY columns (Equality, IN Predicates etc.)
- Queries using selected geo func



- Search Access Path gets updated with data needed for look up
- New updates in table gets automatically updated
- Check progress search_optimization_progress in SHOW TABLES
- Cost for storage and compute

Before SOS

Query should have a column with 100-200k distinct values

Selective filters bring very few records

Query runs at least for few seconds

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

SOS Vs Clustering

SOS applies for all columns
SOS uses addition data structure

Code for SOS :

ALTER TABLE <Table_Name> add search optimization;

Data Loading and Unloading

Snowflake Loading:

Bulk Loading	Continuous Loading
COPY Command	Snowpipe + Copy
Large Data	Incremental Data
User Managed WH	Serverless
64 Days Metadata	14 Days Metadata
Auth Done by Client	Needs Key-Pair Auth with JSON web token

Supported File Type for loading : CSV, TSV, JSON, PARQUET, AVRO, ORC, XML

Bulk Loading:

- Load using Copy Command
- Can also transform the data (Column Re-ordering, Column Omission, Casts, Truncate)
- Auto Duplicate detection
- Stored in the metadata of the target table for 64 days. Available upon completion of the COPY statement as the statement output.
- May not load in files in order

Continuous Loading /Pipes :

- We use Snowpipe
- Loads data in micro batches, within a minute
- Serverless – Managed by Snowflake
- Use options like Stream, Tasks

Example of PIPE:

```
CREATE [ OR REPLACE ] PIPE [ IF NOT EXISTS ] <name>
[ AUTO_INGEST = [ TRUE | FALSE ] ]
[ ERROR_INTEGRATION = <integration_name> ]
[ AWS_SNS_TOPIC = '<string>' ]
[ INTEGRATION = '<string>' ]
[ COMMENT = '<string_literal>' ]
AS <copy_statement>;
```

File Considerations while loading

-The number of load operations that run in parallel cannot exceed the number of data files to be loaded

-Split Large files in smaller ones and combine small ones in large one
-Ideal size 100-250 mb (or larger) in size compressed
-Loading very large file eg. 100 GB or larger, not recommended
-For large Files, carefully use ON_ERROR option
-If load goes beyond 24 hours, then job can be cancelled with no data load
-Consider accumulating file in source at 1 minute frequency

FILE FORMAT:

Supported File Type for loading : CSV (Default),TSV, JSON, PARQUET, AVRO, ORC, XML,CUSTOM

Supported File Type for Unloading : CSV (Default),TSV,JSON,PARQUET

Compression : **AUTO** (Default) | GZIP | BZ2 | BROTLI | ZSTD | DEFLATE | RAW_DEFLATE | NONE

Data can be loaded/unloaded with UI or SQL statement.

File Format definition (Example)

```
CREATE OR REPLACE FILE FORMAT my_csv_ff
TYPE = CSV
FIELD_DELIMITER = ','
SKIP_HEADER = 1
NULL_IF = ('NULL', 'null')
EMPTY_FIELD_AS_NULL = true
COMPRESSION = NONE;
```

Describe FF

```
DESC FILE FORMAT my_csv_ff_new;
```

Show FF

```
SHOW FILE FORMATS;
```

Drop FF

```
DROP FILE FORMAT my_csv_ff_new;
```

FF can be set at COPY INTO, STAGE and TABLE DEFINITION.

Put Command

```
PUT file://<File Path>CSV @~/customers_user/staged AUTO_COMPRESS = FALSE
OVERWRITE = TRUE;
```

COPY INTO example:

```
COPY INTO mytable
FROM s3://mybucket/data/files
STORAGE_INTEGRATION = myint
ENCRYPTION=(MASTER_KEY = 'eSx...')
FILE_FORMAT = (FORMAT_NAME = my_csv_format);
```

Various option with COPY INTO ON_ERROR Options:

```
ON_ERROR = { CONTINUE | SKIP_FILE (DEFAULT IN SNOWPIPE) | SKIP_FILE_<num> |
'SKIP_FILE_<num>%' | ABORT_STATEMENT (DEFAULT IN BULK LOADING) }
SIZE_LIMIT = <num>
RETURN_FAILED_ONLY = TRUE | FALSE (DEFAULT)
```

```
MATCH_BY_COLUMN_NAME = CASE_SENSITIVE | CASE_INSENSITIVE | NONE
(DEFAULT) ---
ENFORCE_LENGTH = TRUE (DEFAULT) | FALSE
TRUNCATECOLUMNS = TRUE | FALSE
FORCE = TRUE | FALSE
PURGE = TRUE | FALSE
```

VALIDATE and VALIDATION_MODE

VALIDATION_MODE = RETURN_n_ROWS | RETURN_ERRORS | RETURN_ALL_ERRORS

Example : COPY INTO customers_named
FROM @internal_named_stage
FILE_FORMAT = my_csv_ff
ON_ERROR = CONTINUE
RETURN_FAILED_ONLY = TRUE;

VALIDATE: Validates the files loaded in a past execution of the COPY INTO <table> command

Example: SELECT * FROM TABLE (VALIDATE (customers_named, JOB_ID => '_last'))

Unloading data :

- Bulk(Table) unloading with COPY INTO <Location>
- For getting single file from multiple files use SINGLE = TRUE, default is FALSE
- Can partition the data by PARTITION BY

GET Command

```
GET @internal_named_stage file ://<File_path>;
```

Empty String and NULL VALUE

Empty strings Zero length and no characters , denoted as ""

NULL values means absence of data, demoted as null

-- **INSERT OVERWRITES** the existing data in the employees table with the new data from the select statement.

```
INSERT OVERWRITE INTO employees SELECT id, name, department FROM employees
WHERE department = 'Engineering';
```

Data Transformations

Estimation Functions:

Distinct Count: Uses **HyperLogLog (HLL) Algo** to count distinct value in a column.

Similarity Estimate: Uses **MinHash Function** to find similarity between 2 datasets.

Frequency Estimate: Uses **Space Saving Algo** to approximate frequency of value in the dataset. Function like: APPROX_TOP_K, APPROX_TOP_K_ACCUMULATE/COMBINE/ESTIMATE

Percentile Estimate: Uses **T-digest Algo** to approximate percentile values in the dataset.

SAMPLING :

FIXED-SIZE

Fixed Numbers of rows are returned

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

Example: `SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER SAMPLE(100 ROWS);`

FRACTION BASED

BERNOULLI | ROW : $(p/100)*n$, Number of rows vary, row wise

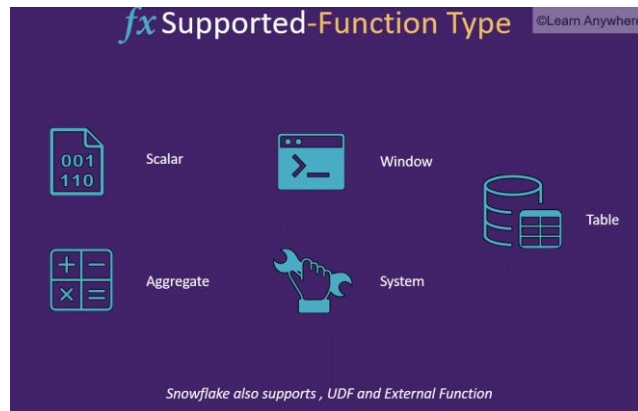
Example: `SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE BERNOULLI (20.3) SEED(10);`

SYSTEM | BLOCK : Faster, Might be biased, block wise

Example: `SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE(1);`

Sampling on Join is possible if, BERNOULLI/ROW BASED, No Seed Used.
SAMPLE and TABLESAMPLE are synonymous.

Supporting Function Types:



Scalar Function: Returns 1 value per invocation.
Example: `SELECT CEIL(135.135);` Returns >> 136

Aggregate Function: Returns 1 value per group of rows.
Example: `SELECT AVG(salary) from <table>;`

Window Function : Aggregate functions that can operate on a subset of rows within the set of input rows.
Example : `SELECT province, store_ID, profit, 100 * RATIO_TO_REPORT(profit) OVER (PARTITION BY province) AS percent_profit FROM store_profit ORDER BY province, store_ID;`

System Function : Control Functions that allows you to execute action in the system.
Example : `SELECT SYSTEM$ABORT_SESSION(1065153868222);`

Table Function : A table function returns a set of rows for each input row. The returned set can contain zero, one, or more rows. Each row can contain one or more columns.
Typically used in FROM clause
It is wrapped TABLE()

- 1-to-N

Functions that return multiple rows for each input, such as record high temperatures for a specified date.

- M-to-N

Functions that depend on multiple input rows to generate each output row, like moving averages of stock prices.

Types : System define (eg. FLATTEN), User Define (eg. UDTF)

External Function: External functions in Snowflake are user-defined functions (UDFs) that call executable code stored and executed outside of Snowflake.

- Remotely executed code is called remote service (i/p JSON, o/p JSON)
- Information sent using proxy service using API Integration
- WH and Data Transfer charges
- Scalar Function, returns 1 row per input row

```
select my_database.my_schema.my_external_function(col1) from table1;
```

Streams:

A stream object records data manipulation language (DML) changes made to tables, including inserts, updates, and deletes, as well as metadata about each change, so that actions can be taken using the changed data. This process is referred to as change data capture (CDC).

Types

- Standard
- Append-only
- Insert-only (External Table)

Supported table types

- Standard tables (Including shared table)
- Directory tables
- External tables
- Underlying tables for a view

Streams goes stale when source tables go out of retention period
Stale stream doesn't track changes
Shall consume the stream within retention period

DATA_RETENTION_TIME_IN_DAYS	MAX_DATA_EXTENSION_TIME_IN_DAYS	Consume Streams in X Days
14	0	14
1	14	14
0	90	90

Check Stream's stale date in column **STALE_AFTER**

Extra column comes with Stream, **METADATA\$ACTION**, **METADATA\$ISUPDATE**
METADATA\$ROW_ID

Stream Code Example:

```
CREATE OR REPLACE STREAM raw_data_stream ON TABLE raw_data;
```

Append only Stream:

```
CREATE OR REPLACE STREAM raw_data_stream2 ON TABLE raw_data append_only=true;
```

Tasks:

A task can execute, Single SQL statement, Call to a stored procedure, Procedural logic using Snowflake Scripting.

Tasks can execute with serverless or user-managed WH

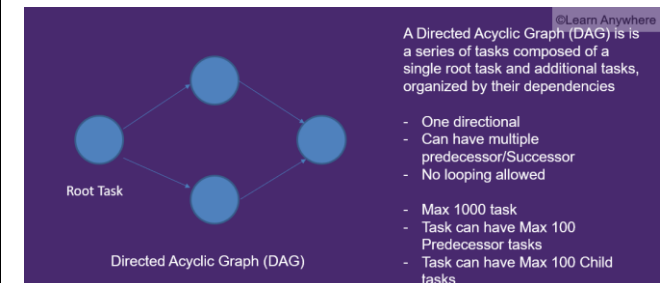
Category	Serverless Task	User-Managed Task
Utilization	When you cant fully utilize a WH	When you can fully utilize the WH
Workload	Predictable load	May have Spike
Scheduling	When Schedule is important	When Schedule is less important
Limitation	Max WH size is XXLARGE Java/Python UDF Scala Stored Procedure	

-Create TASK with User Managed Warehouse (Example)

```
CREATE OR REPLACE TASK TASK_DEMO_U
WAREHOUSE = COMPUTE_WH
SCHEDULE = '1 MINUTE'
AS INSERT <command>;
```

--Create TASK for Serverless (Example)

```
CREATE OR REPLACE TASK TASK_DEMO_S
USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL' --By Default Medium
SCHEDULE = '1 MINUTE'
AS INSERT <command>;
```



For scheduling can also use CRON option

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

©Learn Anywhere

```
CREATE TASK t1
SCHEDULE = 'USING CRON 0 9 * * SUN America/Los_Angeles'
USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
AS <INSERT Command>;
```

Option	Description	Example
Minute	The minute of the hour.	0-59
Hour	The hour of the day.	0-23
Day of Month	The day of the month.	1-31
Month	The month of the year.	1-12
Day of Week	The day of the week.	0-7 (0=Sunday, 1=Monday, etc.)

Semi-Structured Data : Semi-structured data is data that does not conform to the standards of traditional structured data, but contains tags (labels) or other types of mark-up that identify individual, distinct entities within the data.

Snowflake provides native data types (**ARRAY, OBJECT, and VARIANT**)
Snowflake supports querying the data
Supported Data formats (For Load)

- JSON
- Avro
- ORC
- Parquet
- XML

Supported Data formats (For Unload)
- JSON
- Parquet

In a **VARIANT** column, JSON null values are stored as a string containing the word "null"

©Learn Anywhere

Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']
OBJECT(Max 16MB)	A data type used to store an unordered collection of key-value pairs, where each key is a string and each value can be a different data type.	{'name': 'John', 'age': 35, 'isMarried': true}
VARIANT(Max 16MB)	A data type used to store semi-structured data, including JSON documents, XML documents, and other complex data types. VARIANT can store any type of data and allows you to query and manipulate it as needed.	{ 'name': 'John', 'age': 35, 'isMarried': true, 'hobbies': ['reading', 'hiking', 'cooking'], 'address': { 'street': '123 Main Street', 'city': 'Anytown', 'state': 'CA', 'zip': '12345' } }

FLATTEN is used for simple data, **LATERAL FLATTEN** for complex data.

Example of flattening of data:

CREATE OR REPLACE table dump_json as

```
SELECT src:Department::string as dep,  
      Zone.value:SubZone :: string as subzone_col,  
      div.value:Store :: string as store_col,  
      ord.value:SR :: string as sr_col,  
      ord.value:Sales :: string as sales_col
```

```
FROM raw_source,  
      LATERAL FLATTEN(INPUT => SRC:Zone) as Zone,  
      LATERAL FLATTEN(INPUT => zone.value:Division) as div,  
      LATERAL FLATTEN(INPUT => div.value:Orders) as ord;
```

Unstructured data : Unstructured data is information that does not fit into a predefined data model or schema.

Example : PDF, Image, Video, VCF (genomics), KDF (semiconductors), HDF5 (aeronautics) etc.

SF Supports for Unstructured data :

- Securely access data files located in cloud storage.
- Share file access URLs with collaborators and partners.
- Load file access URLs and other file metadata into Snowflake tables.
- Process Unstructured data.

©Learn Anywhere

Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <u>expiration_time</u>
How-to-Fun	BUILD_SCOPED_FILE_URL	BUILD_STAGE_FILE_URL	GET_PREIGNED_URL
Syntax	BUILD_SCOPED_FILE_URL (@<stage_name>, <relative_file_path>)	BUILD_STAGE_FILE_URL (@<stage_name>, <relative_file_path>)	GET_PREIGNED_URL (@<stage_name>, <relative_file_path> , [<expiration_time>])

©Learn Anywhere

SQL Function	Description
GET_STAGE_LOCATION	Returns the URL for an external or internal named stage using the stage name as the input.
GET_RELATIVE_PATH	Extracts the path of a staged file relative to its location in the stage using the stage name and absolute file path in cloud storage as inputs.
GET_ABSOLUTE_PATH	Returns the absolute path of a staged file using the stage name and path of the file relative to its location in the stage as inputs.
GET_PREIGNED_URL	Generates the pre-signed URL to a staged file using the stage name and relative file path as inputs. Access files in an external stage using the function.
BUILD_SCOPED_FILE_URL	Generates a scoped Snowflake-hosted URL to a staged file using the stage name and relative file path as inputs.
BUILD_STAGE_FILE_URL	Generates a Snowflake-hosted file URL to a staged file using the stage name and relative file path as inputs.

GET_PREIGNED_URL and BUILD_SCOPED_FILE_URL are non-deterministic functions; the others are deterministic.

Directory :

- Stores file level meta data about files in stage

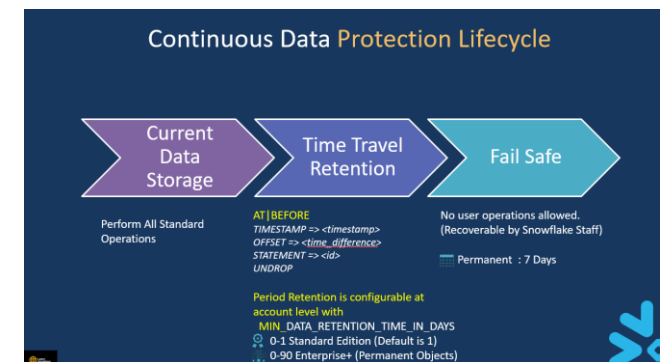
- Implicit object layered on stage
- Can be enabled for external/internal stage
- Stores URL by BUILD_STAGE_FILE_URL
- Can be enabled during stage creation or later (by alter)
- Data can be refreshed manually or automatically (only for cloud i.e external stages)
- No explicit privilege required to access

Example Code :

```
CREATE OR REPLACE STAGE internal_UN_stage  
ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')  
DIRECTORY=(ENABLE=TRUE);
```

Data Protection and Data Sharing

Continuous Data Protection with Snowflake:



DATA ENCRYPTION:

At Rest : Encrypted with AES 256 Bit
At Transit : TLS 1.2

Periodic Key Rotation

- Snowflake auto-rotates the key every 30 days
- Post 30 days, Active Keys get retired

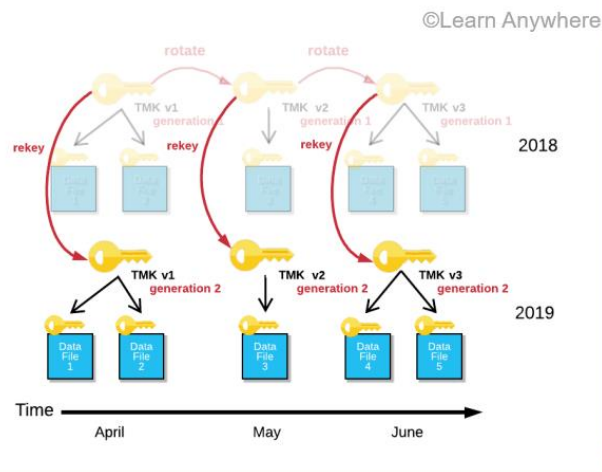
Periodic Rekeying (Enterprise + feature)

- Manual Enablement
- Destroys retired key post one year
- Ensures that data is encrypted with latest tech

Tri-Secret Secure (Business Critical Edition on Higher)

- Combination of CMK and SMK
- Composite Master Key is used
- If CMK is revoked, You cant decrypt data in Snowflake
- Provides 3 levels of data protection: encryption, authentication, and control

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>

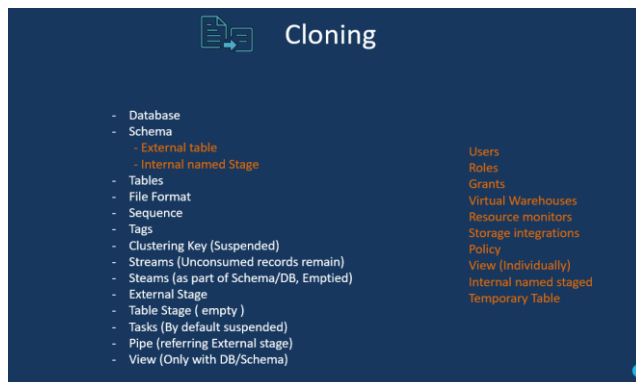


CLONING:

Cloning in Snowflake refers to the process of creating an exact copy of a database, schema, or table. This feature can be used to create a backup of existing data, test new data structures, or create a new environment for development or testing purposes.

Cloning doesn't copy the load metadata, hence any file loaded in original table, can be loaded in cloned table.

Objects which can be cloned and which **can't be cloned** shown below.



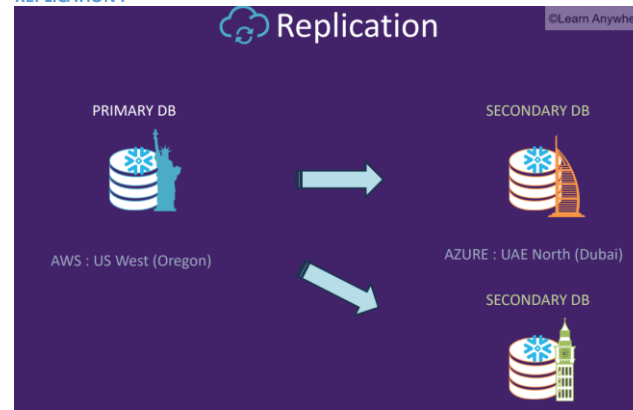
Cloning with Time-Travel is only available for Only available for Database, Schema, Table & Streams

Cloning fails if

- Source in past doesn't exists
- Pipe object with AUTO_INGEST = TRUE recreated

Code : CREATE OR REPLACE TABLE clone_data CLONE sample_data;

REPLICATION :



- Only for DB objects
- Privileges granted on DB are not replicated
- Error when replicating to lower edition (have workaround)
- Can replicate to multiple accounts
- Data transfer and Data Compute cost

Below objects are **not supported** for Replications:

- External tables
- Event tables
- Stages
- Pipes
- DBs created from SHARE

Feature	Standard	Enterprise	Business Critical	VPS
Database replication	✓	✓	✓	✓
Share replication	✓	✓	✓	✓
Replication Group	✓	✓	✓	✓
Account object (other than database and share) replication			✓	✓
Failover Group			✓	✓
Data protected with Tri-Secret Secure			✓	✓

Secure Data Sharing (Not available for VPS): Secure Data Sharing lets you share selected objects in a database in your account with other Snowflake accounts. No data gets copied or replicated.

Secure data sharing can be done with other snowflake account or readers (non-sf) account.

Below objects can be shared:

- Database
- Tables
- External, Dynamic, Apache Iceberg tables
- Secure views
- Secure materialized views
- Secure UDFs
- Model without code (eg CORTEX_FINETUNED)

When data shared with other SF account, consumer pays for the compute.

When data shared with none SF account, provider pays for the compute.

Data Sharing Considerations:

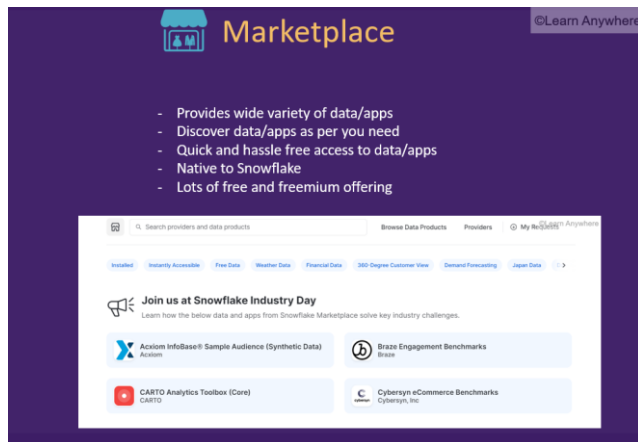
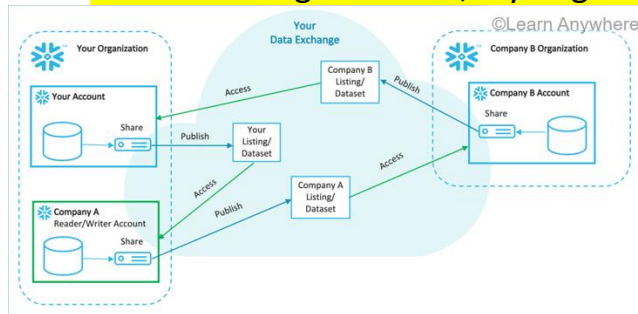
- Shared databases are read-only. Users in a consumer account can view/query data, but cannot insert or update data, or create any objects in the database
- No limit on Number of Shares or Number of accounts it is shared with
- Can not create a clone of a shared database or any schemas/tables in the database
- Not supported, Time Travel for a shared database or any schemas/tables in the database
- Not supported, Editing the comments for a shared database.
- Shared databases and all the objects in the database cannot be re-shared with other accounts
- Shared databases cannot be replicated
- Can consume as many as shares you want but can create only db per share
- Can't share from Business critical to non-Business critical account, can override settings
- Sharing happens between same cloud provider and same region (For cross region, need to replicate first)
- Sharing TO and FROM VPS is not supported

Data Provider : Under Data > Provider Studio

- Need ACCOUNTADMIN or relevant Auth
- Can create and publish listing
- Monitor the performance of listing

Data Exchange: Snowflake support can enable it, can share and control data between selected group of entities.

This Book is created as part of Udemy course Snowflake SnowPro Core Certification COF-C02 Masterclass, please complete the course before reading this book, if you got this file without the course then you can purchase the course here. <https://bit.ly/snowprocrash>



Best wishes for your exam. For any feedback or corrections, you can always reach us at learn.anywhere1@gmail.com